

Java Backend Architecture

Interview Series

Chapter 17.2

Core Workloads

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 17.2 - Core Workloads

Overview

Kubernetes provides several workload resource types for different deployment patterns. Understanding when to use Deployment, StatefulSet, DaemonSet, Job, and CronJob is fundamental for architecting Java backend systems on Kubernetes.

Key Definitions

Pod	Smallest deployable unit; shared network namespace (localhost), shared volumes, init containers.
Deployment	Manages ReplicaSet; rolling update with maxUnavailable/maxSurge; rollback via kubectl rollout undo.
ReplicaSet	Ensures N identical pods running; owned by Deployment; never manage directly.
StatefulSet	Stable hostname (pod-0/pod-1), stable storage (volumeClaimTemplates), ordered startup/shutdown.
DaemonSet	One pod per node: log collection (Fluentd), monitoring (Node Exporter), network plugins.
Job	Runs pods to completion; completions/parallelism; backoffLimit for retries on failure.
CronJob	Schedules Jobs on cron schedule (e.g. 0 */6 * * *); concurrencyPolicy: Forbid/Allow/Replace.
Init Container	Runs to completion before main containers start; used for setup, migrations, wait-for-dependency.
Pod Disruption Budget	minAvailable/maxUnavailable constraints respected during voluntary disruptions (drain, upgrade).
revisionHistoryLimit	Number of old ReplicaSets retained for rollback (default 10); set lower to save etcd space.

Diagram 1: Deployment -> ReplicaSet -> Pod Hierarchy

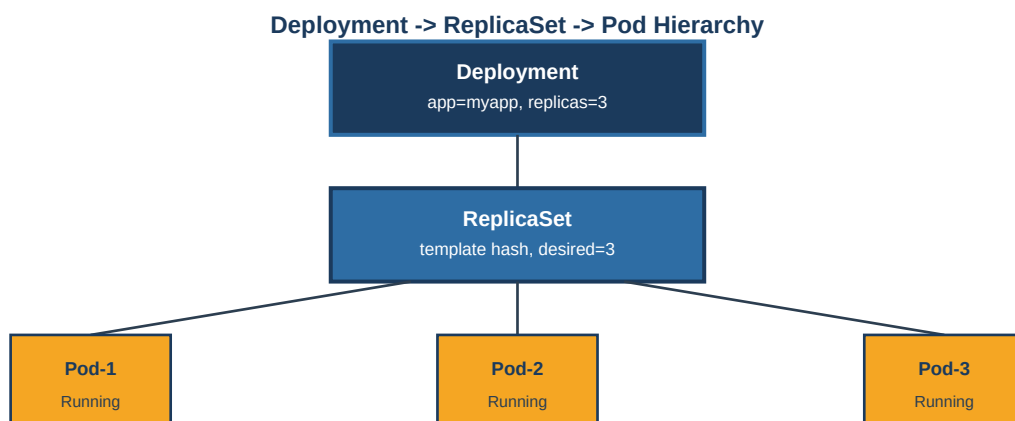
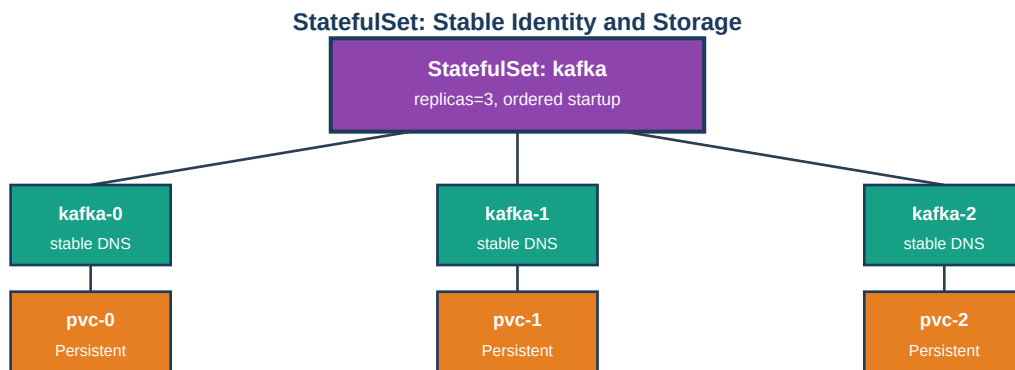


Diagram 2: StatefulSet Stable Identity and Storage



pod-name.service.namespace.svc.cluster.local - stable DNS hostname persists across restarts

Diagram 3: Rolling Update maxUnavailable/maxSurge

Rolling Update: maxUnavailable=1, maxSurge=1



Workload Type Comparison

Workload	Use Case	Identity	Storage	Ordering
Deployment	Stateless web/API	Random pod names	Shared or ephemeral	No ordering
StatefulSet	Kafka, Cassandra, DB	Stable pod-0/pod-1	Per-pod PVC	Ordered start/stop
DaemonSet	Logging, monitoring	Per-node	Host path or none	No ordering
Job	Batch processing	Random	Ephemeral or PVC	Run to completion
CronJob	Scheduled tasks	Random per run	Ephemeral	Cron schedule

Code Examples

Deployment with rolling update strategy:

```
apiVersion: apps/v1 kind: Deployment metadata: name: java-app spec: replicas: 3
revisionHistoryLimit: 5 strategy: type: RollingUpdate rollingUpdate: maxUnavailable: 1
maxSurge: 1 selector: matchLabels: {app: java-app} template: metadata: labels: {app: java-app}
```

```
spec: containers: - name: app image: myrepo/java-app:v2.0 resources: requests: {cpu: 500m,
memory: 512Mi} limits: {cpu: 2, memory: 1Gi} startupProbe: httpGet: {path: /actuator/health,
port: 8080} failureThreshold: 30 periodSeconds: 10 livenessProbe: httpGet: {path:
/actuator/health/liveness, port: 8080} initialDelaySeconds: 0 periodSeconds: 10
readinessProbe: httpGet: {path: /actuator/health/readiness, port: 8080} periodSeconds: 5
```

StatefulSet for Kafka:

```
apiVersion: apps/v1 kind: StatefulSet metadata: name: kafka spec: serviceName: kafka-headless
replicas: 3 podManagementPolicy: OrderedReady selector: matchLabels: {app: kafka} template:
spec: containers: - name: kafka image: confluentinc/cp-kafka:7.5.0 env: - name: POD_NAME
valueFrom: fieldRef: {fieldPath: metadata.name} volumeClaimTemplates: - metadata: name: data
spec: accessModes: [ReadWriteOnce] storageClassName: gp3 resources: requests: {storage: 100Gi}
```

CronJob example:

```
apiVersion: batch/v1 kind: CronJob metadata: name: report-generator spec: schedule: "0 */6 * *
*" concurrencyPolicy: Forbid successfulJobsHistoryLimit: 3 failedJobsHistoryLimit: 1
jobTemplate: spec: backoffLimit: 2 template: spec: restartPolicy: OnFailure containers: - name:
reporter image: myrepo/reporter:latest
```

Interview Q&A; - Core Workloads

Q1. What is a Pod and what makes it the fundamental unit in Kubernetes?

A Pod is the smallest deployable unit in K8s. All containers in a Pod share: the same network namespace (communicate via localhost), the same UTS namespace (hostname), and optionally shared volumes. This co-location model suits tightly coupled processes (app + sidecar proxy). Pods are ephemeral - they are not rescheduled on the same node if deleted; their IP changes on restart.

Q2. When should you use StatefulSet vs Deployment?

Use Deployment for stateless workloads (REST APIs, web apps) where any pod is interchangeable. Use StatefulSet when you need: stable network identity (kafka-0.kafka.ns.svc.cluster.local), stable persistent storage (each pod gets its own PVC from volumeClaimTemplates), ordered startup/shutdown (pod-N-1 must be Running before pod-N starts). Required for Kafka, Cassandra, ZooKeeper, MySQL replication.

Q3. How does rolling update work in a Deployment?

Deployment creates a new ReplicaSet with updated pod template. It scales up the new RS and scales down the old RS in interleaved steps respecting maxUnavailable (max pods unavailable = down from desired) and maxSurge (max pods above desired during update). Example: 3 replicas, maxUnavailable=1, maxSurge=1 allows 4 pods total max, minimum 2 available at all times. Readiness gates must pass before pod counts as available.

Q4. How do you roll back a Deployment?

kubectl rollout undo deployment/app -- reverts to previous revision by scaling up the old ReplicaSet. kubectl rollout undo deployment/app --to-revision=2 -- reverts to specific revision. kubectl rollout history deployment/app -- shows revision history. revisionHistoryLimit controls how many old ReplicaSets are kept (default 10).

Q5. What are init containers and when do you use them?

Init containers run sequentially to completion before any main container starts. Use cases: (1) wait-for-dependency: wait for database to be ready before starting app (using wget/curl check). (2) DB migrations: run Flyway/Liquibase migration before app starts. (3) Configuration setup: fetch secrets from Vault, generate config files. (4) Clone git repo or download artifacts to shared volume.

Q6. What is a DaemonSet and its common use cases?

DemonSet ensures one pod per node (or subset via nodeSelector/tolerations). Common uses: log collection (Fluentd, Filebeat ship node logs to Elasticsearch), monitoring (Prometheus Node Exporter for node metrics), networking (CNI plugin pods, Calico node agent), security scanning (Falco for runtime threat detection). DaemonSet pods are updated via rolling update or OnDelete strategy.

Q7. Explain the difference between Job and CronJob.

Job runs one or more pods to completion - succeeds when N completions reached, fails after backoffLimit retries. parallelism: N allows concurrent pods. completions: M total successes needed. CronJob creates Jobs on a cron schedule (0 */6 *** = every 6 hours). concurrencyPolicy: Forbid (skip if previous still running), Allow (multiple concurrent), Replace (kill old, start new).

Q8. What are the pod lifecycle phases and what triggers each?

Pending: pod accepted by API server but not yet scheduled or images not pulled. Running: pod bound to node, at least one container running. Succeeded: all containers exited with code 0 (batch jobs). Failed: all containers terminated, at least one non-zero exit or killed. Unknown: API server cannot get pod status (node communication failure).

Q9. How does Pod Disruption Budget protect services during maintenance?

PDB defines `minAvailable` (e.g., 2) or `maxUnavailable` (e.g., 1). During voluntary disruptions (`kubectl drain`, rolling node upgrades, Cluster Autoscaler scale-down), the eviction API checks PDB. If evicting a pod would violate the budget, eviction is refused and drain waits. Ensures high availability during planned maintenance. Does not apply to involuntary disruptions (node failure).

Q10. What is `maxSurge` in a rolling update and why is it important?

`maxSurge` allows creating extra pods above the desired count during update - enables zero-downtime updates. Example: 3 replicas, `maxSurge=1` allows 4 pods temporarily. New pods start and become Ready before old pods are terminated. `maxSurge=0` forces in-place replacement (delete one old, create one new). For Java apps, `maxSurge=1` is recommended to maintain full capacity during updates.

Q11. How does `StatefulSet` handle ordered startup and shutdown?

With `podManagementPolicy: OrderedReady` (default): pods start sequentially (pod-0 Running+Ready before pod-1 starts). Shutdown is reverse order (pod-2 deleted first). Parallel: all pods start/stop simultaneously. This ordering is critical for Kafka (broker-0 must be up before broker-1 joins the cluster), ZooKeeper (majority needed before accepting connections), and similar clustered systems.

Q12. What is `volumeClaimTemplates` in `StatefulSet`?

`volumeClaimTemplates` defines a PVC template that K8s automatically creates one per pod: `kafka-data-kafka-0`, `kafka-data-kafka-1`, etc. Each pod mounts its own PVC. When a pod is rescheduled, it reattaches the same PVC (stable storage). PVCs are NOT deleted when `StatefulSet` is deleted (by design) - you must delete PVCs manually to avoid data loss.

Q13. How does a `Deployment` handle readiness during rolling update?

During rolling update, a pod is counted as available only when its `readinessProbe` passes. The rolling update respects `maxUnavailable` based on ready pods. If new pods fail readiness, the rollout stalls (does not proceed but also does not roll back automatically). `kubectl rollout status deployment/app` shows progress. Use `minReadySeconds` to require pods to stay ready for N seconds before counting as available.

Q14. What happens to a pod if the node it runs on fails?

If the node fails (kubelet stops sending heartbeats), the Node controller marks it `NotReady` after `node-monitor-grace-period` (40s). After `pod-eviction-timeout` (default 5 min), pods on the failed node get a taint-based eviction and are deleted. The `Deployment/ReplicaSet` controller then creates replacement pods on healthy nodes. `StatefulSet` pods are also recreated but may wait for persistent volume to be detached from failed node first.

Q15. How do you configure graceful shutdown for a Java Spring Boot app in Kubernetes?

Set `server.shutdown=graceful` in `application.properties` (Spring Boot 2.3+). Set `spring.lifecycle.timeout-per-shutdown-phase=30s`. In pod spec, set `terminationGracePeriodSeconds: 60` and `preStop` lifecycle hook (sleep 5 to let load balancer drain). When pod receives `SIGTERM`: (1) removed from Service endpoints (readiness probe removed), (2) `preStop` hook runs, (3) `SIGTERM` sent to PID 1, (4) Spring drains in-flight requests, (5) `SIGKILL` after grace period.

Q16. What is the `Recreate` deployment strategy and when to use it?

strategy: type: `Recreate` - kills all existing pods before creating new ones. Causes downtime. Use when: new version is incompatible with old version (database schema change), resource constraints prevent running both versions simultaneously, or single-instance apps that cannot run duplicate instances (license servers). For most Java microservices, `RollingUpdate` is preferred.

Q17. How do you perform a blue-green deployment in Kubernetes?

Create two Deployments (blue: current, green: new). Use a Service with selector matching the active deployment (color: blue or color: green). Test green deployment independently. Switch traffic by updating Service selector to color: green. Rollback: change selector back to color: blue. Argo Rollouts provides automated blue-green with analysis and automatic promotion/rollback.

Q18. Explain canary deployments in Kubernetes.

Canary: route small % of traffic to new version. Simple approach: run 1 new pod alongside 9 old pods (Service selects both via shared label) - 10% traffic to new. Better approach: Argo Rollouts with canary steps (setWeight: 10, pause, setWeight: 50, analysis). Istio/Flagger approach: traffic splitting via VirtualService weights - precise % control independent of replica count.

Q19. What are ephemeral containers and when do you use them?

Ephemeral containers are temporary containers added to a running pod for debugging - they do not restart on failure and cannot be added to pod spec. `kubectl debug -it pod/app --image=busybox --target=app-container`
Use cases: debug distroless images (no shell), attach debugging tools without modifying production image, inspect running container filesystem and network.

Q20. How does Kubernetes handle container crashes?

When a container exits non-zero: kubelet restarts it per restartPolicy (Always/OnFailure/Never). Restart backoff: 10s, 20s, 40s, 80s, 160s, max 300s. Pod enters CrashLoopBackOff state. After backoff resets (pod has been running for 10 minutes), timer restarts. Check: `kubectl logs pod/app --previous` (previous container logs), `kubectl describe pod` (event messages).

Q21. What is a sidecar container pattern?

Sidecar: additional container in same Pod sharing network and volumes with main app. Use cases: service mesh proxy (Istio Envoy intercepts all traffic), log shipper (Filebeat reads shared log volume and ships to Elasticsearch), secrets refresher (Vault agent writes refreshed secrets to shared volume), adapter pattern (transforms app metrics format for Prometheus scraping). K8s 1.29+ has native sidecar containers that start before main and stop after main.

Q22. How do you configure resource requests and limits for a Java container?

Memory: request == limit for Guaranteed QoS (no OOMKill surprises). Minimum for Spring Boot: 512Mi request, 1Gi limit. Large services: 2Gi/4Gi. CPU: request = average load (500m), limit = peak (2000m) - allows bursting. JVM flags: `-XX:+UseContainerSupport -XX:MaxRAMPercentage=75` (75% of limit for heap). `-XX:ActiveProcessorCount=2` if CPU limit is 2.

Q23. What is the pod security context?

securityContext at pod level sets defaults for all containers. Key settings: `runAsNonRoot: true` (prevents root containers), `runAsUser: 1000`, `fsGroup: 2000` (volume permissions). Container securityContext: `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem: true`, `capabilities: {drop: [ALL], add: [NET_BIND_SERVICE]}`. Required for PodSecurity restricted standard compliance.

Q24. How do you handle database migrations with Kubernetes Deployments?

Best practice: init container runs migration (Flyway/Liquibase) before app starts. The init container fails the pod if migration fails, blocking app startup. For rolling updates: migrations must be backward-compatible (add column, do not rename/drop). Use expand/contract pattern: Phase 1 (add new column, app reads both), Phase 2 (write to new), Phase 3 (drop old). Alternative: Kubernetes Job for migration before Deployment update.

Q25. What is the difference between Recreate and RollingUpdate strategies?

Recreate: set `spec.strategy.type: Recreate` - terminates ALL old pods before creating new ones. Guaranteed downtime. Use for incompatible version changes or stateful single-instance apps. RollingUpdate (default): gradually replaces old pods with new ones. Controls: `maxUnavailable` (default 25%) and `maxSurge` (default 25%). Zero downtime when readiness probes properly configured.

Q26. How does the kubelet manage container images?

`imagePullPolicy`: Always (always pull, no local cache - for mutable tags like latest), IfNotPresent (use local if exists - for immutable digest-pinned images), Never (only local images). `imagePullSecret` references a Secret with docker credentials. Best practice: pin images by digest (`image@sha256:abc...`) for reproducible deployments. containerd caches layers - only changed layers are pulled.

Q27. What is a Kubernetes Job and how does completionMode work?

Job `completionMode`: NonIndexed (default) - any N pod completions count toward completions target. Indexed - each pod gets a `JOB_COMPLETION_INDEX` env var (0 to completions-1). Use Indexed for parallel batch processing where each worker handles a specific data partition. Java Spring Batch jobs: run multiple partitioned steps as Indexed Job pods for parallel processing.

Q28. How do you implement zero-downtime deployments with readiness probes?

`readinessProbe` ensures new pods are ready before old pods are removed. For Spring Boot: `/actuator/health/readiness` returns 200 only when app is fully initialized (database connections established, caches warmed). `periodSeconds: 5`, `failureThreshold: 3` means a pod is removed from endpoints within 15s of failure. `successThreshold: 1` (default) means one success is enough to mark ready.

Q29. What is podManagementPolicy in StatefulSet?

`OrderedReady` (default): pods created one at a time in order (0, 1, 2...), each must be Running+Ready before next is created. Deletion is reverse order. Ensures cluster formation sequence. Parallel: all pods created/deleted simultaneously. Use when ordering is not needed (e.g., read replicas) for faster scale operations. Changing this field requires deleting and recreating the StatefulSet.

Q30. How do you force a rolling restart of a Deployment?

`kubectl rollout restart deployment/app` - adds annotation `kubernetes.io/change-cause` or `kubernetes.io/restartedAt` timestamp to pod template, triggering a rolling update without changing the actual spec. Useful to: force pod restart to pick up new ConfigMap values (env vars do not auto-update), reload certificates, restart after Secret rotation.

Q31. What is the difference between terminationGracePeriodSeconds and preStop hook?

`terminationGracePeriodSeconds`: total time from SIGTERM to SIGKILL (default 30s). `preStop` lifecycle hook: runs synchronously before SIGTERM is sent to PID 1. The hook execution counts against `terminationGracePeriodSeconds`. Common pattern: `preStop: exec: command: [sleep, "15"]` to give load balancer time to drain connections before app receives SIGTERM. Total time: 15s (sleep) + app shutdown time.

Q32. Explain the pod scheduling with resource requests.

Scheduler uses resource requests (not limits) to find feasible nodes. Allocatable capacity = Node capacity - system reserved - kube reserved. A pod is schedulable if sum of requests fits within node allocatable capacity. Actual usage may exceed requests (up to limits). Over-provisioning resources by setting low requests causes node resource pressure and pod evictions. `kubectl describe node` shows Allocated resources per node.

Q33. What is a namespace and how does it provide isolation?

Namespace provides soft isolation: separate resource names, RBAC scope, ResourceQuota, NetworkPolicy. Not a security boundary (unlike VMs) - privileged pods can escape. Common patterns: per-team (team-a, team-b), per-environment (dev, staging, prod), per-application. kube-system: core components. kube-public: public resources. default: default namespace for user workloads.

Q34. How do you configure a Horizontal Pod Autoscaler?

kubectl autoscale deployment java-app --cpu-percent=70 --min=3 --max=20 or YAML: apiVersion: autoscaling/v2, kind: HorizontalPodAutoscaler, spec.metrics with type: Resource, resource.name: cpu, target.averageUtilization: 70. Requires metrics-server. Scale-up threshold: 110% of target. Scale-down: conservative to prevent thrashing (5 min window).

Q35. What is the significance of labels and selectors?

Labels are key-value pairs on objects. Selectors filter objects by labels. Service selector determines which pods receive traffic. Deployment selector must match pod template labels (immutable after creation). Use consistent labeling: app, version, component, tier, environment. kubectl get pods -l app=java-app,version=v2 - select by multiple labels.

Q36. How does Kubernetes handle image pull secrets?

imagePullSecrets references a Secret of type kubernetes.io/dockerconfigjson. Create: kubectl create secret docker-registry regcred --docker-server=... Add to pod spec: spec.imagePullSecrets: [{name: regcred}] or patch default ServiceAccount: kubectl patch serviceaccount default -p imagePullSecrets: [{name: regcred}] to apply to all pods in namespace.

Q37. Explain the difference between kubectl delete pod and kubectl drain.

kubectl delete pod: deletes specific pod; Deployment controller creates replacement immediately. kubectl drain node: cordon (marks unschedulable) + evicts all pods respecting PDBs and grace periods. kubectl cordon node: just marks unschedulable, does not evict existing pods. kubectl uncordon node: marks schedulable again. Use drain before node maintenance.

Q38. What is a Job completions and parallelism?

completions: 10, parallelism: 3 - run 3 pods concurrently, 10 total completions needed. Each pod runs once; if pod fails, new pod starts (up to backoffLimit). For Java batch: use Indexed Jobs with spring-batch partitioned steps - each pod processes its index partition. Work queue pattern: each pod processes items from queue until empty.

Q39. How does Kubernetes manage pod disruptions during node maintenance?

kubectl drain --ignore-daemonsets --delete-emptydir-data node-1: evicts pods respecting PDBs, waits if eviction would violate minAvailable. Draining a node with PDB minAvailable=2 and only 2 running pods will block until another pod is scheduled elsewhere. Use --timeout to avoid indefinite waiting. Node taints node-role.kubernetes.io/unschedulable:NoSchedule during drain.

Q40. What is the difference between a Deployment and a bare Pod?

Bare Pod is not managed by any controller - if it fails or its node fails, it is not recreated. Deployment manages lifecycle through ReplicaSet: ensures desired replica count, rolling updates, rollback history. Always use Deployment (or StatefulSet/DaemonSet) instead of bare pods in production. Bare pods are only useful for one-off debugging or testing.

Q41. How does autoscaling interact with PodDisruptionBudgets?

Cluster Autoscaler respects PDBs when scaling down nodes - will not evict pods if it would violate minAvailable. HPA scale-down: replicas are managed by Deployment rolling update but HPA can trigger scale-down to minimum replicas. If PDB minAvailable equals desired replicas, HPA cannot scale down. Set

PDB minAvailable slightly below desired minimum for proper autoscaling.

Q42. What is a Job activeDeadlineSeconds?

activeDeadlineSeconds: maximum time a Job can run. If exceeded, all pods are terminated and Job status set to Failed. Use to prevent stuck jobs running indefinitely. For Java batch jobs with SLA: set activeDeadlineSeconds to 2x expected duration. Combine with ttlSecondsAfterFinished: 3600 to auto-clean completed/failed Jobs.

Q43. How does the readiness gate work?

ReadinessGate adds custom conditions to pod readiness evaluation. Pod is not ready until both all containers pass readiness probes AND all ReadinessGates conditions are True. Use case: service mesh (Istio sets IngressReady condition when Envoy sidecar is ready), AWS Load Balancer Controller (sets target group health condition). Ensures external dependencies are ready before pod receives traffic.

Q44. What is an emptyDir volume and when is it useful?

emptyDir: {} creates an ephemeral volume that lives as long as the pod. Shared between containers in the same pod. Use cases: init container writes config to emptyDir, main container reads it; sidecar log shipper reads app logs from shared emptyDir; tmpfs mount (medium: Memory) for in-memory temp storage. Data lost when pod is deleted/rescheduled.

Q45. How does a StatefulSet handle pod failure and rescheduling?

When a StatefulSet pod fails, the controller creates a new pod with the same name (kafka-1) on any available node. The new pod reattaches the same PVC (kafka-data-kafka-1). If the old node is partitioned (not confirmed dead), K8s may not reschedule to avoid split-brain (two pods with same name and PVC). Use force-delete carefully: `kubectl delete pod kafka-1 --force --grace-period=0` (ensure old pod truly dead).

Q46. What is a CronJob suspend field?

spec.suspend: true prevents new Jobs from being created by the CronJob schedule. Existing Jobs continue running. Use to temporarily disable a CronJob during maintenance or when underlying service is unavailable. Re-enable: `kubectl patch cronjob report-gen -p {"spec":{"suspend":false}}`. startingDeadlineSeconds: 100 - if CronJob misses scheduled time by >100s, skip that run.

Q47. How does Kubernetes handle container image caching?

containerd caches image layers on the node. Pull policy: IfNotPresent checks if image exists locally by tag - if yes, skip pull (even if remote changed). Always: always pull manifest, check if local matches (faster than full re-pull if layers cached). For production: use digest-pinned images (image@sha256:abc) with imagePullPolicy: IfNotPresent for immutable, reproducible deployments.

Q48. What is a topologyKey in PodAntiAffinity?

topologyKey defines the topology domain for spreading. `kubernetes.io/hostname`: pods spread across different nodes (each pod on unique node). `topology.kubernetes.io/zone`: pods spread across zones. `topology.kubernetes.io/region`: pods spread across regions. Required in requiredDuringScheduling to ensure pods on different nodes/zones for HA. Java microservices: zone-spread for multi-AZ HA.

Q49. How do you configure resource limits for init containers?

Init containers have separate resource requests/limits from main containers. For scheduling, K8s uses max(init container requests, sum of main container requests). Each init container runs exclusively (not simultaneously), so only the max init request matters. Set requests for init containers that run DB migrations (may need more CPU for migration) separate from the main app runtime requirements.

Q50. What is the significance of finalizers in StatefulSets?

StatefulSet uses `kubernetes.io/statefulset-pod` finalizer on pods to prevent premature deletion. This ensures orderly teardown: pod is not fully deleted until StatefulSet controller removes the finalizer after proper shutdown. Prevents data corruption from abrupt pod termination. If a StatefulSet pod is stuck in Terminating, check and manually remove the finalizer only after ensuring data safety.

Q51. Explain how multiple containers in a Pod communicate.

Containers in same Pod share network namespace - communicate via `localhost:port`. No inter-container DNS resolution needed. Shared volumes allow file-based IPC. Main app container serves on `:8080`, sidecar Nginx proxy listens on `:80` and proxies to `localhost:8080`. Envoy sidecar intercepts traffic on `:15001` (outbound) and `:15006` (inbound) via iptables rules.

Q52. How does Kubernetes handle node resource pressure?

kubelet monitors resource usage via cAdvisor. Under memory pressure: evicts BestEffort pods first, then Burstable. Under disk pressure (nodefs): evicts pods with highest disk usage. Under PID pressure: evicts pods with most processes. Node marks itself with conditions: `MemoryPressure`, `DiskPressure`, `PIDPressure`. Scheduler avoids scheduling new pods on pressured nodes. Java pods should use Guaranteed QoS (`requests==limits`) to avoid eviction.

Q53. What is kubectl rollout status and how does it work?

`kubectl rollout status deployment/app` blocks until rollout completes or times out. Returns exit code 0 on success, 1 on failure. Use in CI/CD pipelines to verify deployment success. Internally polls Deployment conditions: `Available`, `Progressing`. `Progressing` condition deadline exceeded = deployment stuck. `kubectl rollout pause/resume` allows pausing mid-rollout for canary inspection.